

Probabilistic Programming

Marius Popescu

popescunmarius@gmail.com

2025 - 2026

Mock Exam | Discussion | Questions

Mock Exam

Question # (0.5 point)

Let us assume a probability distribution for count data. Which of the following type of distribution we would choose for modelling:

- ☐ Bernoulli distribution
- ☐ Exponential distribution
- ☒ Poisson distribution

Question # (0.5 point)

From the following distribution which one is a conjugate prior for the Bernoulli distribution:

- ☐ Gamma distribution
- ☒ Beta distribution
- ☐ Uniform distribution on $(0, 1)$

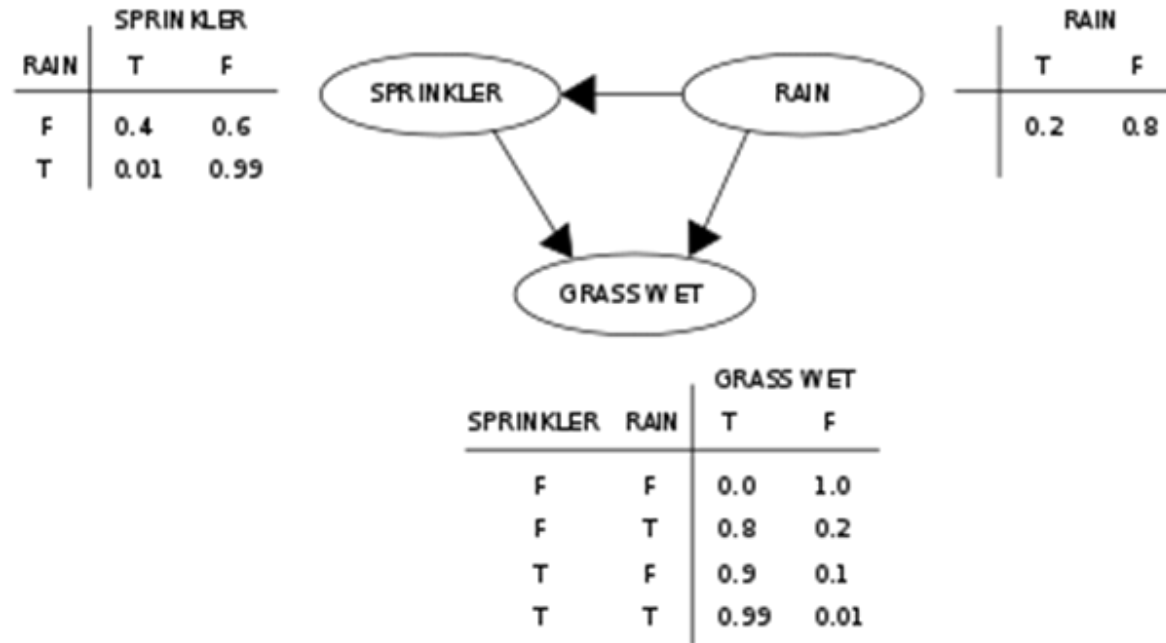
Problem # (0.5 point)

Write two different PyMC stochastic variables that can represent a discrete random variable X that takes value in $\{0, 1, 2, 3, 4, 5\}$. Which one is the most uninformative?

- `X = pm.DiscreteUniform("X", lower = 0, upper = 5)`
- `X = pm.Categorical("X", p = [0.1, 0.1, 0.1, 0.2, 0.2, 0.3])`
- `DiscreteUniform` (or both if the values of p are equal)

Problem # (1.5 points)

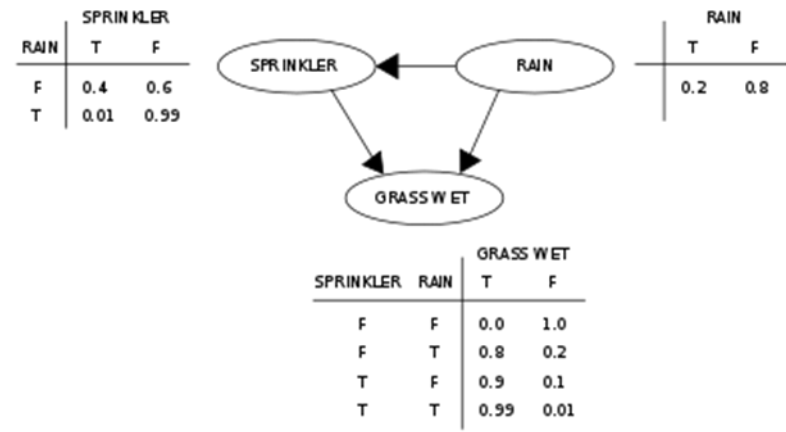
Consider the following Bayesian network:



Write a PyMC Model that models this Bayesian network.

Solution

- with `pm.Model()` as `model`:
- `R = pm.Bernoulli("R", 0.2)`
- `p_S = pm.math.switch(R, .01, .4)`
- `S = pm.Bernoulli('S', p_S)`
- `p_W = pm.math.switch(S, pm.math.switch(R, .99, .9), pm.math.switch(R, .8, 0.0))`
- `W = pm.Bernoulli("W", p_W)`



Problem # (2.5 points)

Suppose I have a box of dice that contains a 4-sided die, a 6-sided die, an 8-sided die, a 12-sided die, and a 20-sided die. If you have ever played Dungeons & Dragons, you know what I am talking about.

Suppose that someone selects a die from the box at random, roll it, and get a 6. What is the probability that he rolled (selected) each die?

Write a (complete) PyMC program to infer the probability of selecting each die.



D20



D12



D8



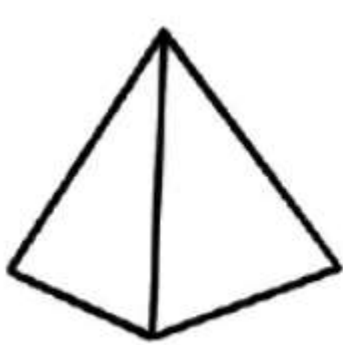
D6



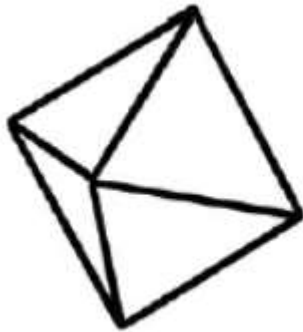
D4

NB: Platonic Solids

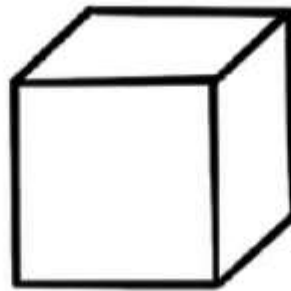
The ancient Greeks would go on to prove that there are only five possible dice that you can carve with all faces made up of the same two-dimensional symmetrical shape. This proof is the highlight of Euclid's great text, the *Elements*.



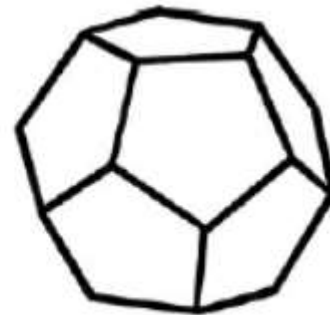
Tetrahedron



Octahedron



Cube



Dodecahedron



Icosahedron

NB: Long Dice (any number of faces)

Long dice are dice, often roughly right prisms designed to land on any of several marked lateral faces, but neither end. Landing on end may be rendered very rare simply by their small size relative to the faces, by the instability implicit in the height of the dice, and by rolling the long dice along their axes rather than tossing. Many long dice provide further insurance against landing on end by giving the ends a rounded or peaked shape, rendering such an outcome physically impossible (at least on a flat solid surface).

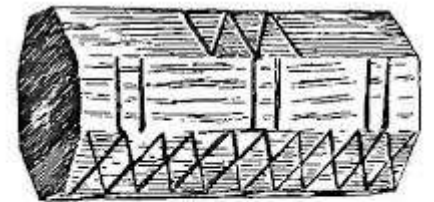


FIG. 85.—LONG LAWRENCE. ALMOND-BURY, ENGLAND.

Specimen reproduced from description given by Mrs. Gomme. Museum of Archaeology, Univ. of Penna. No. 18,257.

Solution

$X = 6$

```
with pm.Model() as model:
```

```
    D = pm.Categorical("D", p = [1.0 / 5.0, 1.0 / 5.0, 1.0 / 5.0, 1.0 / 5.0, 1.0 / 5.0], initval = 3)
```

```
    obs = pm.CustomDist(  
        'obs',  
        D,  
        logp=obs_logp,  
        observed = x  
    )
```

Solution

```
def obs_logp(value, D):  
  
    return pm.math.switch(pm.math.eq(D, 0),  
        pm.math.switch(pm.math.le(value, 4), pm.math.log(1.0 / 4.0), -np.inf),  
        pm.math.switch(pm.math.eq(D, 1),  
            pm.math.switch(pm.math.le(value, 6), pm.math.log(1.0 / 6.0), -np.inf),  
            pm.math.switch(pm.math.eq(D, 2),  
                pm.math.switch(pm.math.le(value, 8), pm.math.log(1.0 / 8.0), -np.inf),  
                pm.math.switch(pm.math.eq(D, 3),  
                    pm.math.switch(pm.math.le(value, 12), pm.math.log(1.0 / 12.0), -np.inf),  
                    pm.math.switch(pm.math.eq(D, 4),  
                        pm.math.switch(pm.math.le(value, 20), pm.math.log(1.0 / 20.0), -np.inf),  
                        -np.inf)  
                    )  
                )  
            )  
        )
```

Solution

```
with model:
    step = pm.CategoricalGibbsMetropolis(vars=[D])
    trace = pm.sample(400000, step=step, tune=100000, return_inferencedata=False, chains=1)

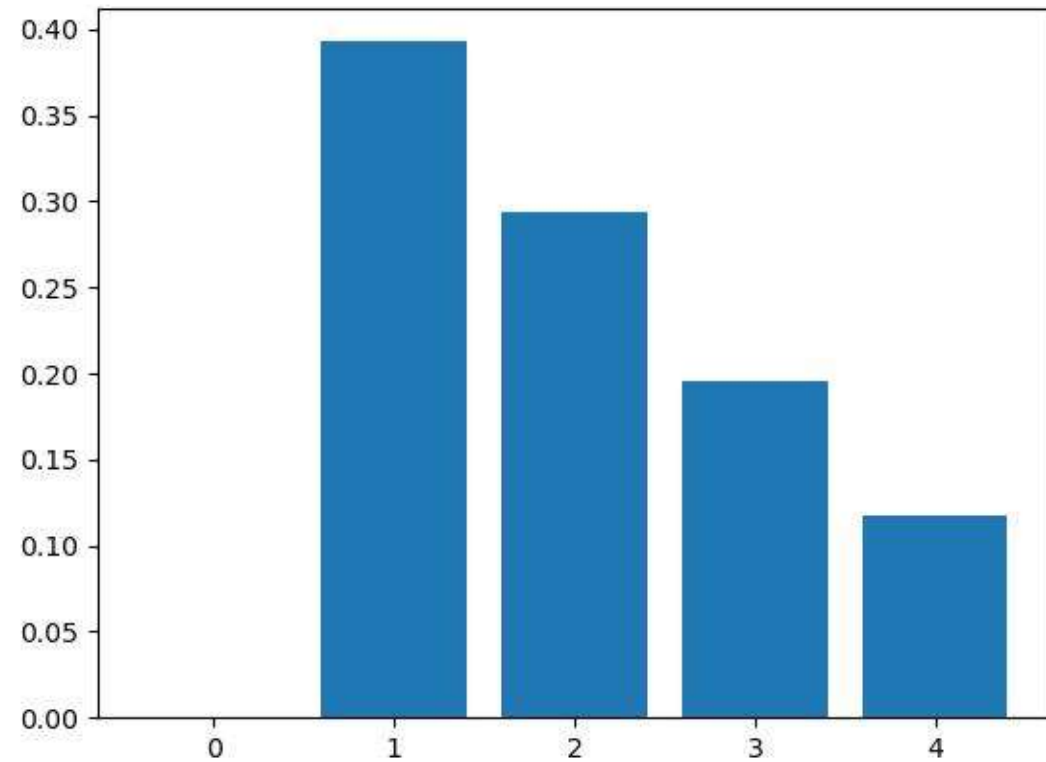
D_samples = trace["D"]

D_samples = np.bincount(D_samples, minlength = 5)
D_samples = D_samples / 400000

print(D_samples)

plt.bar(np.arange(5), D_samples)
plt.show()
```

Results



[0. 0.3929 0.2938 0.1956 0.1175]

Exact (Theoretical) Solution

Let's say we are interested in the probability of a specific die, for example D6 when we observed a roll of 6.

According to the Bayes rule:

$$P(D6|x = 6) = \frac{P(x = 6|D6) * P(D6)}{P(x = 6)}$$

$$P(x = 6|D6) = \frac{1}{6}$$

$$P(D6) = \frac{1}{5} \quad \text{uninformative discrete uniform prior}$$

$$P(x = 6) = \frac{1}{5} * \left(0 + \frac{1}{6} + \frac{1}{8} + \frac{1}{12} + \frac{1}{20} \right) = \frac{17}{200}$$

So:

$$P(D6|x = 6) = \frac{\frac{1}{6} * \frac{1}{5}}{\frac{17}{200}} = \frac{20}{51} = 0.3921$$

Problem # (3.5 points)

Liar's dice is a game that has origins in ancient Peru and has been around since at least the 1500s. The game received a fresh burst of interest in the late 1980s when commercial versions have become widely available. The game makes a prominent appearance in the movie *Pirates of the Caribbean: Dead Man's Chest*, when Will Turner plays against his father, Bootstrap Bill, and Davy Jones.

Each player uses a cup and five normal 6-sided dice in order to play. Once one player is chosen to go first, each player rolls their dice and keeps their dice concealed from the other players using the cup. At this point, the starting player makes a bid reflecting the quantity of a certain die face appearing among all players' dice. A common rule that is added to the game to make it slightly more exciting is that ones are wild (that is, they match whatever die face is the current bid), and ones may not be bid. For instance, the first player, in a game with seven players, may see their dice faces as (3, 4, 1, 6, 1) and decide to bid that there are at least 13 fours at the table. The next player (to their left) may choose to:



Problem # (3.5 points)

- challenge that bid if they think that there are fewer than 13 fours at the table,
- raise that bid with a higher bid of their own.

When challenged, if the bid is indeed correct (of, say, at least 13 fours (and ones) at the table), the challenger loses one of their dice; if the bid is wrong, the bidder loses the die instead.

For raising, the new bid must increase either the quantity of dice involved in the bid, or the numeric value of the die face involved, or both. For the initial bid of 13 fours, for example, 13 fives or 13 sixes would be an appropriate raise, as would 14 twos. However, 13 twos would not be a valid bid.

The opportunity to raise or challenge goes around the table clockwise until someone decides to challenge. At that point, after the outcome is decided, the player to the left of the initial bidder starts the next round, and play continues until all players but one are eliminated (have no dice remaining).

Problem # (3.5 points)

Question

What are the probabilities for the quantity of named faces at Liar's Dice, Given the fact that we know the faces of our dice?

How can we determine this? For our group of seven players, there are a total of 35 dice at the table. Of course, we already know what our 5 dice are. What will be the probability of having at the table a specific number of dice?

Write a (complete) PyMC program that given the number of players ($n_{p1} = 7$) and the number of a particular face (the actual face doesn't matter) in our hand ($m_{y1f} = 3$), the program infer the probability $P(k)$, the probability of having exactly k dice that match either a one or the named value, for each possible k .

Solution 1

```
npl = 7
mynf = 3

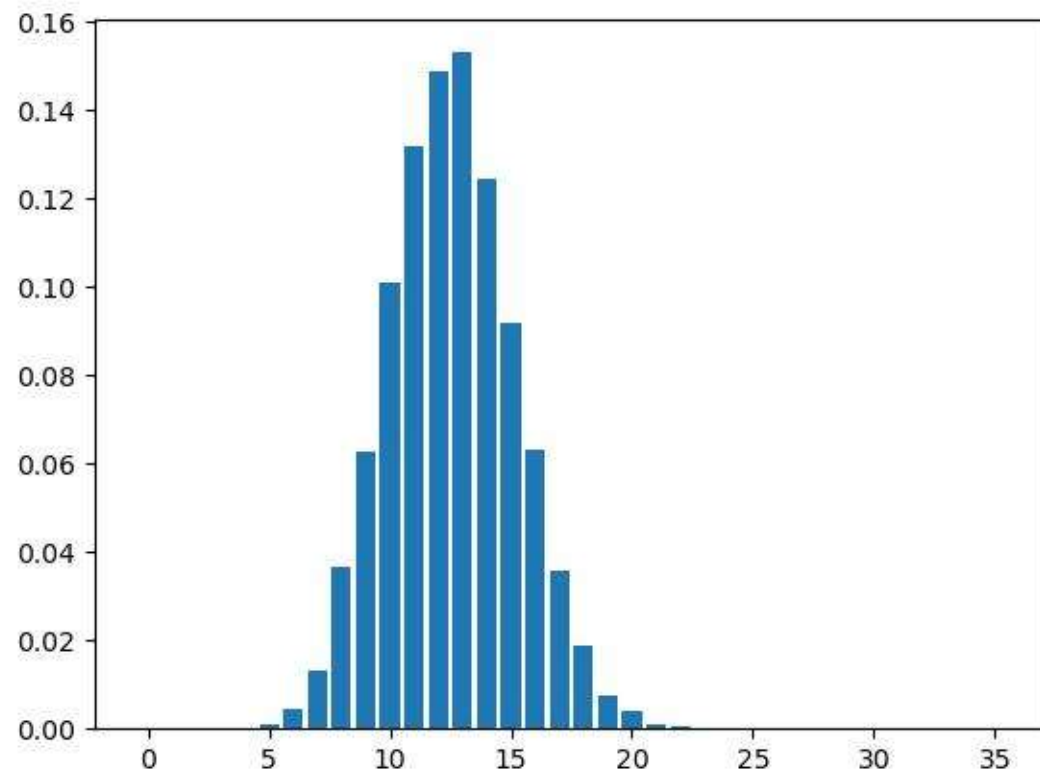
with pm.Model() as liardice_model:
    Nf = pm.Binomial('Nf', n = 5 * npl, p = 1 / 3)
    obs = pm.Binomial('obs', n = Nf, p = 1 / npl, observed = mynf)

with liardice_model:
    trace = pm.sample(40000, tune=10000, return_inferencedata=False, chains=1)

Nf_samples = trace["Nf"]
Nf_samples = np.bincount(Nf_samples, minlength = 5 * npl + 1)
Nf_samples = Nf_samples / 40000

plt.bar(np.arange(5 * npl + 1), Nf_samples)
plt.show()
```

Results



Solution 2: Faster, Better, More Accurate

```
np1 = 7
mynf = 3

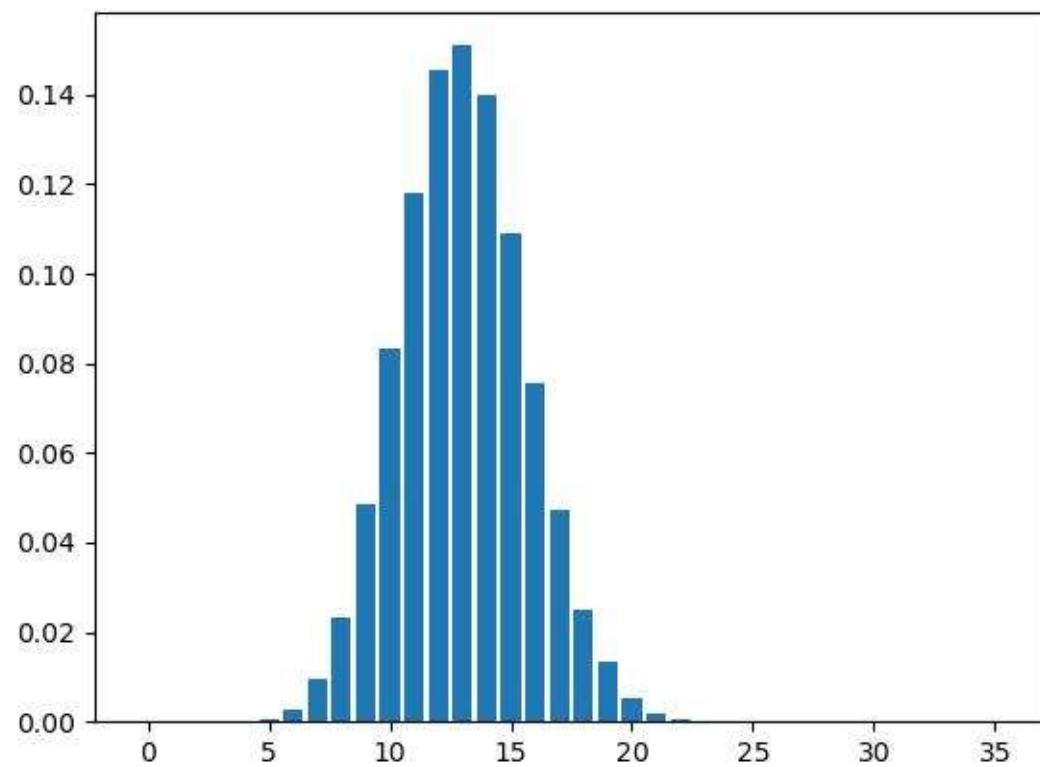
with pm.Model() as liardice_model:
    Nf = pm.Binomial('Nf', n = 5 * (np1 - 1), p = 1 / 3)

Nf_samples = 3 + pm.draw(Nf, draws = 40000)

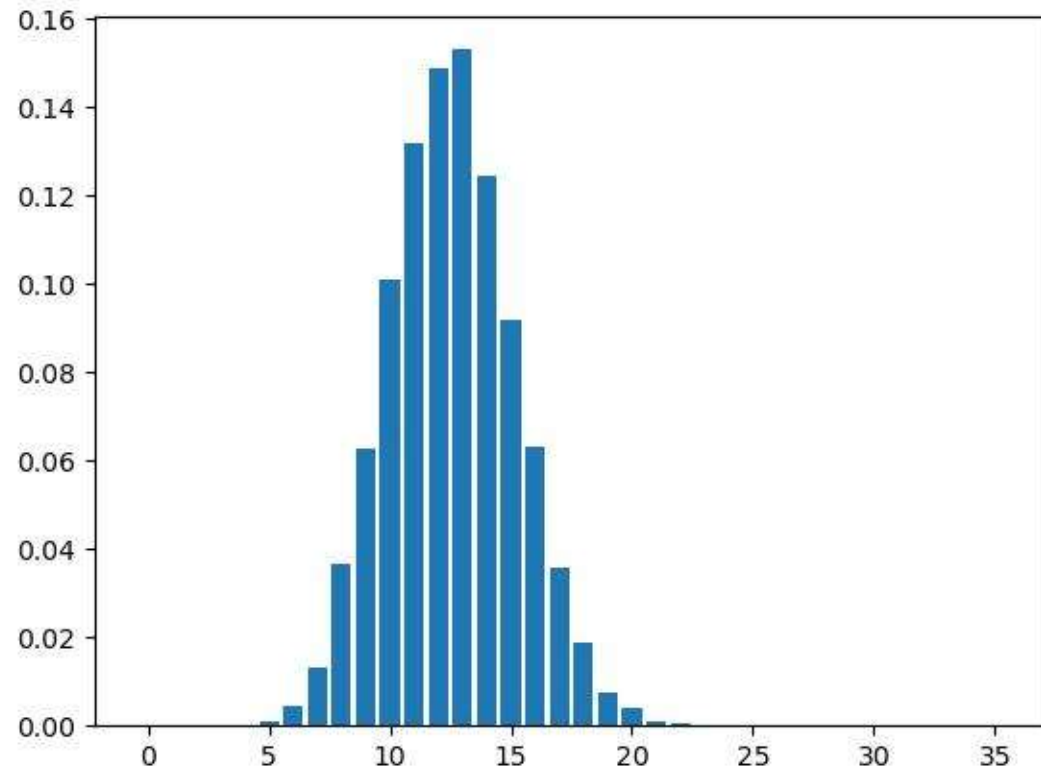
Nf_samples = np.bincount(Nf_samples, minlength = 5 * np1 + 1)
Nf_samples = Nf_samples / 40000

plt.bar(np.arange(5 * np1 + 1), Nf_samples)
plt.show()
```

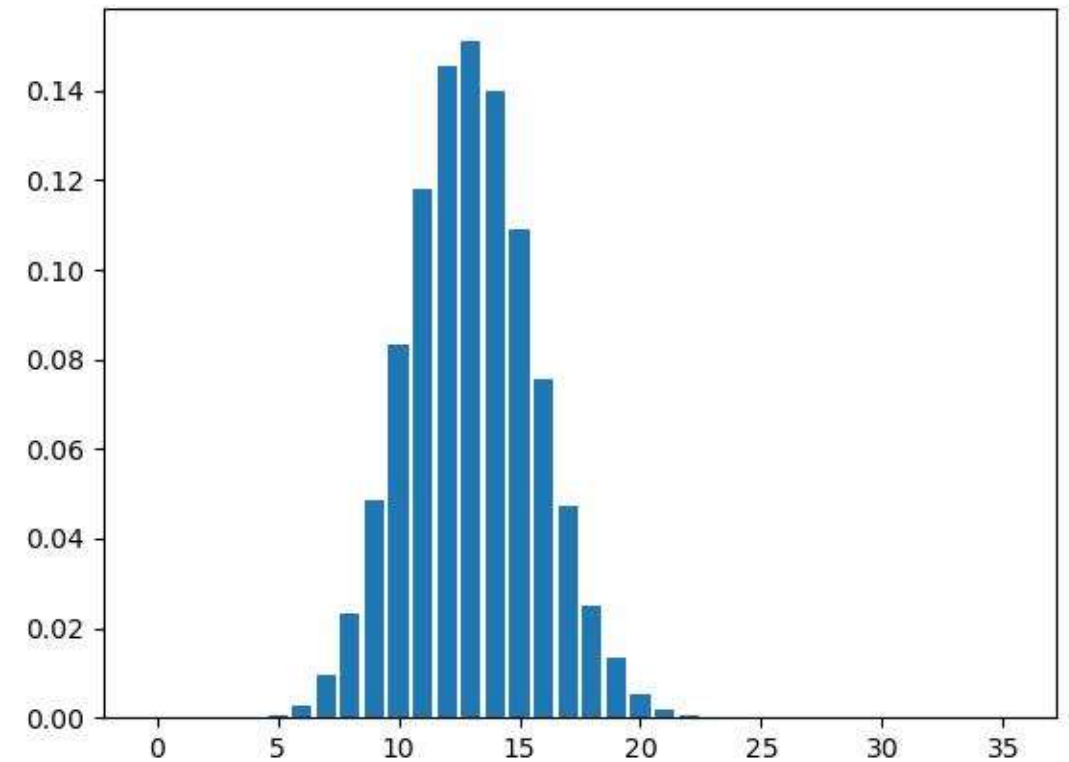
Results



Comparing the Two Results



Solution 1



Solution 2

Let's rewarded ourselves for our effort



OFFICIAL CLIP

Discussion | Questions